

PRÁCTICA DEVOPS

PLAN DEVOPS DEL PROYECTO MÓVIL EVACUA

INGENIERÍA EN DESARROLLO Y GESTIÓN DE SOFTWARE

Presentan:

Gabino Reyes García
José Felipe García Luna
Mariana Guadalupe Cano Márquez
Karol Ximena González López

Grupo:

DS02SV-25

Materia:

Gestión del Proceso de Desarrollo de Software

Docente:

Saldaña Benítez Héctor

Tema:

Planeación del desarrollo móvil con enfoque DevOps

**SAN JUAN DEL RÍO, QRO.
SEPTIEMBRE 2026**



Contenido

- 1. Resumen del caso y objetivos
- 2. Alcance, equipo y arquitectura
- 3. Backlog inicial y Sprint 0
- 4. Ciclo DevOps aplicado
- 5. Versionamiento y ramificación
- 6. Repositorio e integración continua
- 7. Estrategia de pruebas
- 8. Despliegue y liberación
- 9. Monitoreo y métricas
- 10. Riesgos y mitigación
- 11. Runbook de fallas comunes
- 12. Herramienta de comunicación
- 13. Evidencias, README y demostración
- 14. Conclusiones
- 15. Referencias

1. Resumen del caso y objetivos

EVACUA es una aplicación móvil en Flutter acompañada por una plataforma web administrativa. Atiende la dificultad de consultar, durante una emergencia, información clara sobre rutas de evacuación, zonas restringidas, puntos de reunión y equipos cercanos. Sus usuarios iniciales son estudiantes, docentes, trabajadores, visitantes, brigadistas y responsables de seguridad de un edificio piloto. La primera versión utilizará planos simplificados, rutas previamente autorizadas y códigos QR para identificar una zona; no pretende calcular rutas en tiempo real ni sustituir la señalización, los simulacros o las instrucciones de las brigadas. El valor esperado consiste en concentrar información que hoy se encuentra distribuida entre planos, señalética, inventarios y procedimientos, y hacerla consultable desde un teléfono, incluso con conectividad limitada. El plan DevOps establece cómo organizar el trabajo, verificar cambios, distribuir versiones de prueba, observar estabilidad y convertir la retroalimentación en nuevas tareas.

Objetivos

- Construir un prototipo móvil verificable mediante un flujo de trabajo compartido, automatizado y trazable.
- Aplicar control de versiones, revisión por pull request, integración continua y liberaciones semánticas.
- Definir pruebas, distribución alfa y beta, monitoreo, respuesta a fallas y aprendizaje posterior a cada versión.
- Seleccionar una herramienta de comunicación que apoye al equipo sin reemplazar a GitHub como fuente de evidencia técnica.

Indicadores de éxito del prototipo

Indicador	Métrica	Meta inicial
KPI 1	Localización de ruta	Al menos 80 % de participantes identifica la ruta indicada en 30 segundos o menos después de escanear un QR.
KPI 2	Lectura de QR	Al menos 95 % de códigos válidos se reconoce correctamente en las condiciones del piloto.
KPI 3	Estabilidad	Al menos 99 % de sesiones beta termina sin cierre inesperado, medido con Crashlytics.

Criterio de seguridad. EVACUA es un apoyo informativo. Una ruta solo puede publicarse después de revisión del responsable del inmueble; la aplicación debe mostrar que prevalecen los protocolos oficiales y las indicaciones del personal autorizado.

2. Alcance, equipo y arquitectura

Alcance del primer incremento

El prototipo cubrirá un edificio o una sección del campus. Permitirá consultar instrucciones por tipo de emergencia, identificar una zona mediante QR, visualizar un plano simplificado, mostrar rutas preconfiguradas, localizar recursos y marcar manualmente una zona como disponible o bloqueada. El cálculo automático con grafos, el posicionamiento interior, los sensores IoT y las notificaciones en tiempo real quedan fuera del primer incremento porque requieren infraestructura y validación adicional.

Organización del equipo

Integrante	Responsabilidad inicial	Evidencia principal
Gabino Reyes García	Arquitectura y datos	Modelo del inmueble, API y demo QR.
José Felipe García Luna	DevOps e integración	Repositorio, ramas, CI y liberaciones.
Mariana Guadalupe Cano Márquez	Aplicación y pruebas	Interfaz Flutter, pruebas widget y cobertura.
Karol Ximena González López	Producto y documentación	Backlog, criterios, board y guía de comunicación.

La asignación sirve para iniciar Sprint 0; no impide que los integrantes revisen código o participen en otras tareas. Cada elemento debe tener una persona responsable, una incidencia y una evidencia de cierre.

Arquitectura de alto nivel

Se propone separar interfaz, estado y acceso a datos siguiendo la guía de arquitectura de Flutter. Las vistas y view models forman la capa de interfaz, mientras que repositorios y servicios concentran el acceso a API, archivos y caché [1]. Esta separación facilita pruebas y permite sustituir datos simulados por servicios reales sin reescribir todas las pantallas.

Códigos QR y caché local apoyan la consulta móvil

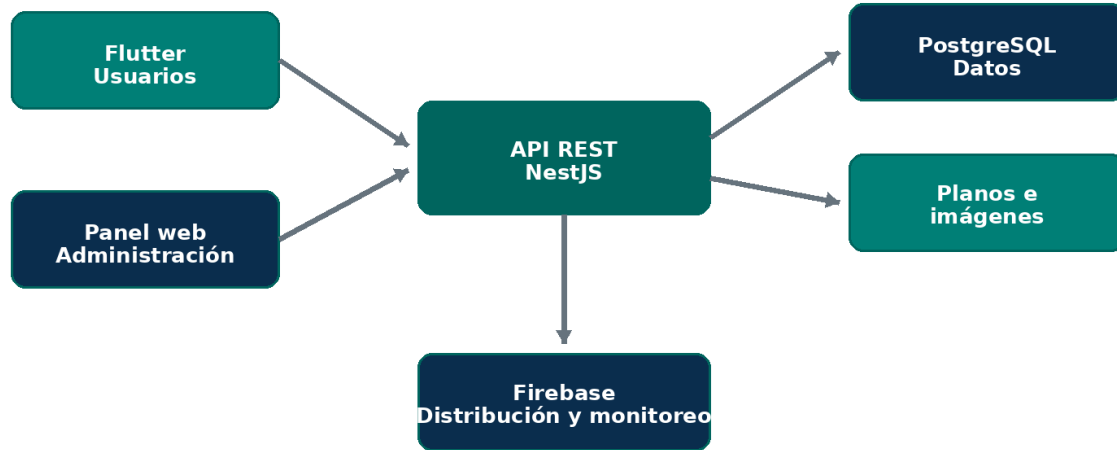


Figura 1. Arquitectura general propuesta para EVACUA.

Dependencia	Uso previsto
Flutter	Aplicación móvil y caché local.
NestJS	API REST y control de permisos.
PostgreSQL	Inmuebles, zonas, rutas, equipos y estados.
Almacenamiento de objetos	Planos e imágenes versionadas.
Firebase	Distribución alfa/beta, Crashlytics y Remote Config.
GitHub	Código, incidencias, board, CI y releases.

3. Backlog inicial y Sprint 0

Las historias siguen el criterio INVEST: deben ser independientes en lo posible, negociables, valiosas, estimables, pequeñas y verificables. La prioridad MoSCoW distingue lo indispensable del prototipo y evita ampliar el alcance antes de validar la propuesta.

ID	Historia resumida	MoSCoW	Pts.	Criterio de aceptación
US-01	Consultar instrucciones por emergencia	Must	3	Se muestran incendio, sismo y fuga con texto autorizado.
US-02	Identificar zona mediante QR	Must	5	QR válido carga zona; QR inválido informa el error.
US-03	Visualizar plano del piso	Must	5	Plano correcto, leyenda visible y ampliación disponible.
US-04	Consultar ruta al punto de reunión	Must	8	Parte de zona actual y evita zonas bloqueadas.
US-05	Localizar equipo de emergencia	Should	5	Filtra por piso y tipo; muestra ubicación y estado.

ID	Historia resumida	MoSCoW	Pts.	Criterio de aceptación
US-06	Consultar información sin conexión	Should	8	Conserva la última versión sincronizada de datos esenciales.
US-07	Registrar inmuebles, niveles y zonas	Must	8	Valida campos y registra autor del cambio.
US-08	Configurar rutas, salidas y puntos	Must	8	Origen y destino válidos; asociación por emergencia.
US-09	Bloquear o habilitar una zona	Must	5	Requiere permiso, registra fecha y actualiza rutas.
US-10	Confirmar llegada al punto de reunión	Could	5	Solicita consentimiento y evita exposición pública.

Definición de Sprint 0

Duración propuesta: dos semanas. El objetivo es disponer de un entorno reproducible, una aplicación inicial, reglas de colaboración, integración continua, backlog priorizado y una demostración con datos simulados. Sprint 0 no se considera terminado por crear carpetas; debe comprobar que una contribución puede recorrer incidencia, rama, pull request, CI, revisión y merge.

ID	Actividad	Responsable propuesto	Criterio de hecho
S0-01	Repositorio, ramas y protección de main	José Felipe	Ramas y reglas visibles.
S0-02	Aplicación Flutter y navegación base	Mariana	App abre y muestra pantalla inicial.
S0-03	Workflow de CI	José Felipe	Formato, análisis y pruebas en verde.
S0-04	Arquitectura y modelo inicial	Gabino	Diagrama y entidades revisados.
S0-05	Historias y board	Karol	Historias priorizadas en columnas.
S0-06	Servidor y canales	Karol	Normas fijadas en anuncios.
S0-07	Prueba widget y cobertura	Mariana	Cobertura mínima de 70 %.
S0-08	QR simulado	Gabino	Identificador abre una zona de prueba.

Tablón de trabajo

GitHub Projects se utilizará porque relaciona el board con incidencias y pull requests, permite campos personalizados, vistas tipo tabla, tablero o roadmap y automatizaciones de estado [4]. Las columnas serán Backlog, To-Do, In-Progress, Review y Done. Cada historia tendrá prioridad, puntos, iteración, tipo y responsable. Se limitará el trabajo en proceso a una o dos tareas por integrante para reducir cambios simultáneos sin terminar.

4. Ciclo DevOps aplicado

El ciclo conecta la planeación con la operación. Cada fase produce un artefacto revisable; por ello, una actividad no se considera terminada únicamente porque el código funciona en una computadora. La retroalimentación de pruebas, estabilidad y usuarios vuelve al backlog.

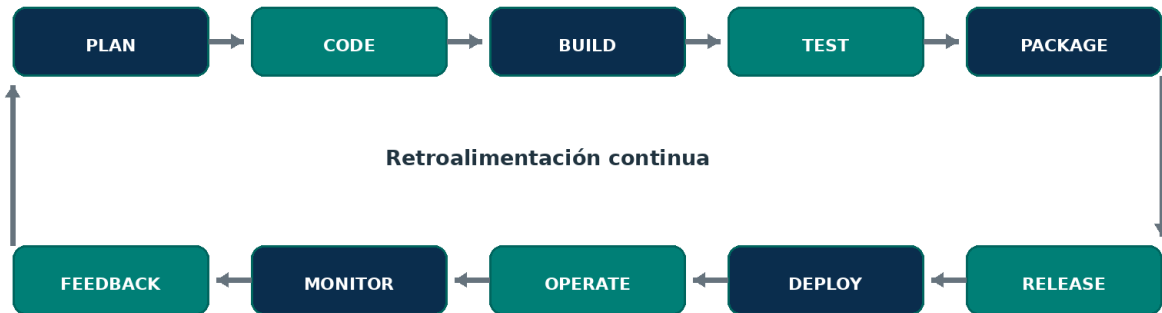


Figura 2. Ciclo DevOps propuesto para EVACUA.

Fase	Objetivo	Herramientas	Artefactos	Criterio de hecho
Plan	Acordar valor y alcance.	GitHub Projects	Historias, criterios, riesgos.	Historia priorizada y estimada.
Code	Implementar cambios revisables.	Git, VS Code, Flutter	Rama, commits y pull request.	Un revisor y convención válida.
Build	Comprobar dependencias y compilación.	GitHub Actions, Flutter	Registro de job y dependencias.	Proceso sin errores.
Test	Detectar defectos antes de integrar.	flutter test, analyze	Cobertura y reportes.	Checks verdes y cobertura $\geq 70\%$.
Package	Crear paquete identificable.	Flutter, Gradle, Fastlane	APK/AAB/IPA firmado.	Nombre y versión coinciden.

Fase	Objetivo	Herramientas	Artefactos	Criterio de hecho
Release	Autorizar una versión.	GitHub Releases	Tag, notas y checklist.	Tag SemVer y aprobación.
Deploy	Entregar a un ambiente.	Firebase App Distribution	Versión alfa o beta.	Instalada por probadores.
Operate	Mantener versión disponible.	Firebase Console	Configuración y accesos.	Responsables y rollback listos.
Monitor	Observar estabilidad y uso.	Crashlytics, Analytics	Paneles, alertas y SLI.	SLO revisado semanalmente.
Feedback	Convertir evidencia en trabajo.	Issues, encuestas, retrospectiva	Defectos y acciones.	Cada hallazgo tiene decisión.

GitHub Actions automatiza workflows dentro del repositorio y puede ejecutar tareas de CI/CD en runners hospedados [3]. Firebase App Distribution acepta APK, AAB e IPA y se integra con CLI, Gradle o fastlane, lo que permite distribuir versiones previas desde un job posterior [8].

5. Versionamiento y ramificación

Git flow ligero

Git representa una rama como un apuntador ligero a un commit, por lo que crear ramas cortas permite aislar trabajo y revisarlo antes de integrarlo [5]. EVACUA utilizará main como línea estable, develop como integración, feature/* para cambios ordinarios y hotfix/* para fallas urgentes. Las ramas de función deben vivir pocos días y enlazar una incidencia.

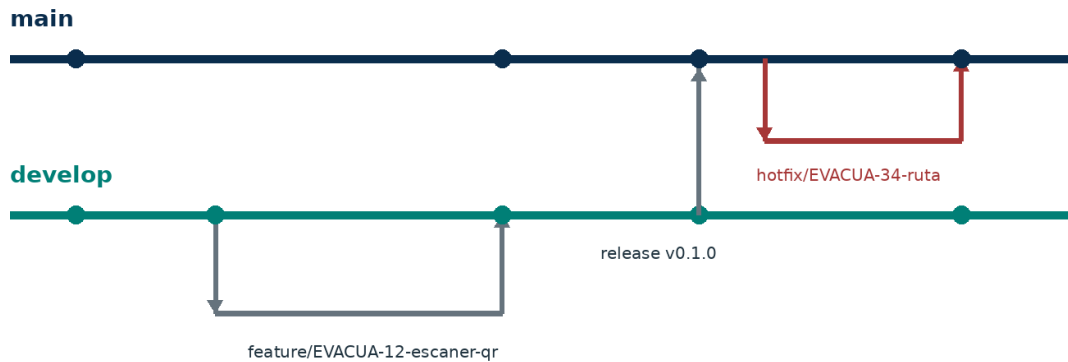


Figura 3. Estrategia de ramificación ligera de EVACUA.

Rama	Propósito	Regla de integración
main	Estable, protegida y asociada a releases.	PR aprobado, CI verde y sin defectos críticos.
develop	Integración del siguiente incremento.	PR aprobado y CI verde.
feature/EVACUA-ID-descripcion	Una historia o tarea; nace de develop.	PR hacia develop y cierre de incidencia.
hotfix/EVACUA-ID-descripcion	Corrección urgente; nace de main.	PR a main, tag patch y réplica en develop.

Commits, pull requests y tags

Conventional Commits organiza el mensaje como tipo, alcance opcional y descripción; distingue funciones, correcciones y cambios incompatibles [6]. Se aceptarán feat, fix, docs, test, refactor, ci, build y chore. Ejemplos:

```
feat(qr): identificar zona desde código válido
fix(routes): excluir zonas bloqueadas
test(map): cubrir selección de punto de reunión
ci(actions): validar cobertura mínima
```

Convención propuesta

Cada pull request debe vincular una incidencia, explicar el objetivo, incluir evidencia, actualizar pruebas y recibir una revisión de alguien distinto al autor. Los checks de formato, análisis, pruebas y cobertura son obligatorios. Se aplicará squash merge para conservar un historial legible.

Semantic Versioning usa MAJOR.MINOR.PATCH: los cambios incompatibles incrementan MAJOR, la funcionalidad compatible incrementa MINOR y las correcciones compatibles incrementan PATCH [7]. Mientras el prototipo permanezca en desarrollo inicial se utilizarán versiones 0.y.z. El primer tag será v0.1.0; una beta podrá identificarse como v0.1.0-beta.1.

6. Repositorio e integración continua

Estructura base

```

EVACUA-DevOps/
├── .github/
│   ├── workflows/flutter_ci.yml
│   ├── ISSUE_TEMPLATE/
│   └── PULL_REQUEST_TEMPLATE.md
├── app/
│   ├── lib/main.dart
│   ├── test/widget_test.dart
│   └── pubspec.yaml
├── ci/check_coverage.py
├── docs/
├── CONTRIBUTING.md
└── README.md

```

Estructura incluida en la entrega

Flujo mínimo de CI

El workflow se activa en push a develop y en pull requests hacia main o develop. Descarga dependencias, comprueba el formato, ejecuta análisis estático, corre pruebas con cobertura y rechaza porcentajes inferiores a 70 %. La cobertura inicial no demuestra por sí sola calidad, pero evita integrar código sin una base verificable.

```

on:
  push:
    branches: [develop]
  pull_request:
    branches: [main, develop]

steps:
  - uses: actions/checkout@v4
  - uses: subosito/flutter-action@v2
    with: { channel: stable, cache: true }
  - run: flutter pub get
  - run: dart format --output=none --set-exit-if-changed lib test
  - run: flutter analyze
  - run: flutter test --coverage
  - run: python ../ci/check_coverage.py coverage/lcov.info 70

```

Extracto del workflow incluido

Criterios para merge y release

Evento	Condiciones
Merge a develop	Historia aceptada, revisión, CI verde, sin secretos y documentación actualizada.
Merge a main	Alcance de versión aprobado, regresión superada y artefacto candidato identificado.
Release	Tag SemVer, notas, artefacto firmado, distribución a grupo correcto y rollback definido.

Las credenciales de firma, Firebase y tiendas se almacenarán como secrets de repositorio o ambiente, nunca en YAML o en archivos versionados. GitHub permite crear secretos a nivel repositorio, ambiente u organización y recomienda reducir el uso de credenciales de larga duración cuando sea posible [11].

7. Estrategia de pruebas

Flutter distingue pruebas unitarias, de widgets y de integración. Las unitarias aíslan funciones o clases; las pruebas widget verifican interfaz e interacción; las de integración comprueban una parte amplia de la aplicación en un dispositivo o emulador [2]. EVACUA combinará estas capas porque una ruta correcta exige tanto lógica válida como una presentación comprensible.

Tipo	Alcance	Momento	Criterio de salida
Unitarias	Selección de rutas, filtrado por zona, validación de QR y caché.	En cada PR.	100 % de reglas críticas y conjunto general ≥ 70 % de líneas.
Widget	Pantallas, estados de carga, errores, leyenda y accesibilidad.	En cada PR.	Sin desbordes; mensajes y controles localizables.
Integración	QR $\rightarrow$ zona $\rightarrow$ plano $\rightarrow$ ruta; bloqueo de zona; modo sin conexión.	Diaria y antes de beta.	Flujos críticos aprobados en Android.
Usabilidad	Comprensión de instrucciones y tiempo para localizar ruta.	Al cierre del incremento.	≥ 80 % completa la tarea en ≤ 30 s.
Seguridad	Autorización, almacenamiento, red y manejo de datos.	Antes de beta y release.	Sin hallazgos altos abiertos; revisión basada en MASVS [13].
Regresión	Funciones Must y defectos corregidos.	Antes de cada release.	Cero defectos críticos y altos.

Criterios de entrada y salida

Entrada. Historia con criterios claros, datos de prueba, ambiente disponible, riesgos identificados y código integrado en una rama revisable.

Salida. Pruebas ejecutadas, evidencia enlazada, defectos clasificados, checks en verde y aceptación del Product Owner o responsable designado.

Datos y casos críticos

- QR válido, vencido, ilegible y correspondiente a otro inmueble.
- Zona disponible, bloqueada y sin ruta alterna autorizada.
- Sin red, sincronización interrumpida y datos locales desactualizados.
- Usuario de consulta frente a responsable autorizado.
- Plano ampliado, orientación vertical u horizontal y texto accesible.

8. Despliegue y liberación

Ambiente	Canal	Cuándo	Controles
Desarrollo	Emulador y dispositivos del equipo.	Cada PR o cambio local.	Datos simulados y logs de depuración.
Alfa	Firebase App Distribution, grupo interno.	Al terminar una historia integrada.	APK no productivo, notas y feedback.
Beta	Firebase App Distribution, usuarios piloto.	Cuando Must críticos están completos.	Crashlytics activo y consentimiento.
Producción	Google Play; App Store cuando exista cuenta y equipo iOS.	Después de aceptación y revisión institucional.	AAB/IPA firmado y checklist aprobado.

App Distribution facilita que equipos de prueba reciban versiones tempranas y admite automatización mediante consola, CLI, fastlane o Gradle [8]. La producción se separará de los ambientes previos para evitar mezclar datos, probadores y credenciales.

Promoción y rollback

- El mismo artefacto candidato debe promoverse; no se recompila código distinto después de aprobar pruebas.
- Una beta se publica a un grupo pequeño y se amplía solo si no aumenta la tasa de fallos.
- Si aparecen cierres inesperados o información incorrecta, se detiene la distribución y se vuelve a la última versión estable.
- Las rutas e instrucciones requieren aprobación funcional independiente del proceso técnico de publicación.

Feature flags

Remote Config se utilizará únicamente en fases posteriores para habilitar funciones no críticas de manera gradual. Firebase permite usar valores como feature flags, comparar estabilidad y detener un rollout [10]. No se empleará un flag remoto para alterar silenciosamente una ruta de evacuación; esos cambios deben conservar autorización y trazabilidad.

9. Monitoreo y métricas

Crashlytics agrupa cierres inesperados y aporta contexto para priorizar problemas de estabilidad en Flutter [9]. Los indicadores operativos se complementarán con métricas de producto. Un SLI representa el nivel de servicio observado y un SLO fija el objetivo que se revisará; Google SRE recomienda usar estas medidas para decisiones de confiabilidad basadas en datos [12].

SLI	Cálculo	SLO inicial	Fuente
Sesiones sin crash	Sesiones sin cierre / sesiones totales	$\geq 99\%$ semanal	Crashlytics
Disponibilidad de API	Solicitudes correctas / solicitudes totales	$\geq 99.5\%$ en piloto	Logs y monitor API
Latencia de sincronización P95	Tiempo de respuesta del 95 % de sincronizaciones	≤ 2 s	APM o logs
Lectura válida de QR	Lecturas correctas / QR válidos probados	$\geq 95\%$	Pruebas y analítica
Actualidad de datos	Zonas publicadas dentro del tiempo acordado	100 % de cambios críticos revisados	Bitácora y auditoría

Alertas y revisión

- Alerta inmediata cuando una versión beta incrementa los crashes o bloquea el flujo QR-ruta.
- Revisión semanal de estabilidad, latencia, fallas de sincronización y comentarios de probadores.
- Vincular cada anomalía con versión, dispositivo, ambiente e incidencia; no conservar datos personales innecesarios.
- Registrar acciones en Feedback y priorizarlas en el siguiente refinamiento del backlog.

10. Riesgos y planes de mitigación

ID	Riesgo	Probabilidad / impacto	Mitigación
R1	Ruta o zona desactualizada	Alta / Alta	Aprobación del responsable, vigencia, auditoría y rollback de configuración.
R2	Conectividad insuficiente	Media / Alta	Caché local, aviso de última sincronización y selección manual de zona.
R3	QR dañado o inaccesible	Media / Media	Códigos redundantes, mantenimiento y alternativa por edificio, nivel y zona.
R4	Exposición de datos o credenciales	Media / Alta	Mínimo de datos, roles, cifrado, secrets y revisión MASVS.
R5	Falla introducida por una liberación	Media / Alta	CI, beta gradual, Crashlytics, checklist y retorno a versión estable.
R6	Dependencia excesiva de la app	Media / Alta	Mensajes claros, capacitación y conservación de señalética y protocolos físicos.

El riesgo principal no es únicamente técnico: una aplicación de apoyo puede inducir confianza indebida si la información parece oficial pero no está actualizada. Por ello, la publicación de rutas y bloqueos exige responsables identificados, historial de cambios y vigencia visible.

Control de seguridad móvil

La revisión se basará en categorías de OWASP MASVS relacionadas con almacenamiento, criptografía, autenticación, red, plataforma, código, resiliencia y privacidad [13]. Para Sprint 0 el control mínimo consiste en excluir secretos del repositorio, separar permisos, evitar datos personales innecesarios y documentar el tratamiento de información antes de habilitar cuentas reales.

11. Runbook de fallas comunes

Falla	Acción inmediata	Escalamiento	Cierre
Build o CI roto	Revisar el primer paso fallido; reproducir con la misma versión estable de Flutter; corregir en rama; no omitir checks.	Autor del PR; DevOps si afecta al workflow.	Ejecución verde y causa registrada.
Credencial expirada	Pausar deploy; rotar secret en el ambiente; comprobar permisos; revocar la credencial anterior.	DevOps y propietario de Firebase/tienda.	Prueba de autenticación y fecha de rotación.
Cuota de API o Firebase	Confirmar panel y alcance; limitar reintentos; habilitar caché; avisar a probadores; solicitar aumento solo con autorización.	Backend y DevOps.	Consumo normal y ticket documentado.
Aumento de crashes	Detener rollout; identificar versión y dispositivos; volver a versión estable o desactivar función no crítica.	Líder de release y desarrollador responsable.	Crash-free recuperado y prueba de regresión.
Ruta o zona incorrecta	Retirar la configuración, mostrar instrucción segura, avisar al responsable del inmueble y conservar evidencia.	Responsable funcional; prioridad máxima.	Dato corregido, aprobado y auditado.

Formato de incidente

INC-AAAA-NNN | Severidad | Ambiente | Versión
 Impacto observado:
 Hora de inicio y detección:
 Acción inmediata:
 Responsable y próxima actualización:
 Evidencia enlazada:
 Causa y acción preventiva al cierre:

Plantilla operativa

Los incidentes que afecten rutas, instrucciones o autorización se clasifican como críticos aunque no provoquen un cierre de la aplicación. La recuperación técnica no completa el incidente hasta comprobar que la información visible es correcta.

12. Herramienta de comunicación

Matriz comparativa

Se compararon funciones disponibles para un equipo universitario pequeño. La escala va de 1 (limitación importante) a 5 (muy adecuado). Los puntajes consideran principalmente planes gratuitos o acceso educativo y deben revisarse si cambian las licencias.

Criterio	Slack	Teams	Discord	Google Chat
Canales e hilos	5	4	5	4
Reuniones y calendario	2	5	4	4
Llamadas	2	5	5	4
Compartir pantalla	2	5	5	4
Bots y automatización	5	4	5	3
Integraciones de desarrollo	5	4	4	4
Permisos	4	5	5	4
Seguridad y compliance	4	5	3	4
Historial gratuito	2	3	5	4
Costo educativo	3	4	5	5
Facilidad móvil	5	4	5	5
Grabaciones nativas	1	2	1	3
TOTAL	40	50	52	48

Slack organiza bien los hilos e integraciones, pero su versión gratuita limita el acceso a los 90 días más recientes y elimina datos de más de un año; además, las conversaciones grupales por huddle no están disponibles en el plan gratuito [14], [15]. Teams destaca en reuniones, seguridad e integración con Microsoft, aunque la versión gratuita limita reuniones grupales a 60 minutos y 100 participantes [16]. Google Chat integra espacios e hilos y resulta conveniente cuando todo el equipo trabaja en Google Workspace [19].

Dictamen

Se selecciona Discord para el prototipo de EVACUA. Los tres criterios decisivos son: acceso gratuito sin depender de una licencia institucional; canales de texto, voz, video y pantalla compartida disponibles para todo el equipo; y facilidad para recibir notificaciones automatizadas. Discord permite organizar servidores por temas y sus canales de voz admiten video y pantalla compartida [17]. Sus webhooks reciben mensajes automáticos desde servicios como GitHub y CircleCI [18], por lo que pueden informar sobre pull requests y CI. El principal límite es que no ofrece grabación nativa y su enfoque de cumplimiento es menor que el de Teams. Se mitigará usando actas breves, consentimiento cuando se utilice otra herramienta de grabación, roles con privilegios mínimos, autenticación de dos factores para administradores y la regla de que GitHub conserva las decisiones técnicas. Si la universidad confirma licencias Microsoft 365 y exige grabación o controles formales, Teams será la alternativa de migración.

Configuración y canales

Canal	Uso
#anuncios	Decisiones, entregas y cambios de alcance. Solo responsables publican.
#dev	Flutter, API, arquitectura y pull requests.
#qa	Casos, defectos, evidencia y resultados de CI.
#ops	Versiones, Firebase, monitoreo e incidentes sin secretos.
#dudas	Preguntas con identificador de historia o incidencia.
#random	Convivencia; no se registran decisiones del proyecto.
Daily EVACUA	Canal de voz para dailies, revisión y demostraciones.

Guía de comunicación del equipo

Normas y tiempos de respuesta

- Usar un hilo por tema y comenzar con el identificador, por ejemplo [US-02] o [INC-2026-001].
- Registrar en GitHub toda decisión que modifique alcance, criterios, código, ambiente o versión.
- Usar @equipo solo para bloqueos, cambios de horario o incidentes que afecten a todos.
- No compartir contraseñas, tokens, llaves de firma, datos personales ni enlaces privados en canales públicos.
- Una duda ordinaria se responde en ocho horas dentro del horario acordado; un bloqueo activo se reconoce en dos horas.
- Fuera del horario acordado no se exige respuesta, salvo una liberación programada y aceptada por el equipo.
- La reunión termina con acuerdos, responsables y fecha. El resumen se publica en #anuncios y enlaza las incidencias.

Formato para dudas

[US-02] Lectura de QR en Android
 Contexto: rama y versión.
 Intenté: pasos realizados.
 Resultado: mensaje o comportamiento observado.
 Esperaba: resultado según criterio.
 Evidencia: captura o log sin secretos.
 Bloqueo: sí/no y fecha necesaria.

Mensaje de soporte

Formato para incidentes

[INC-2026-001] SEV-1 | beta | v0.1.0-beta.1
 Impacto: la ruta no aparece después de escanear.
 Inicio/detección: hora y zona de prueba.
 Acción: distribución pausada.
 Responsable: nombre. Próxima actualización: hora.
 Issue: enlace.

Mensaje en #ops

Integraciones mínimas

- Notificaciones de pull requests y CI mediante webhook de GitHub en #dev y #qa.
- Enlace fijado al GitHub Project y al documento del plan en #anuncios.
- Calendario compartido para daily, revisión, retrospectiva y demostración.
- Alertas de Crashlytics en #ops cuando se habilite la beta.

13. Evidencias, README y demostración

Contenido del paquete de repositorio

La entrega incluye una carpeta lista para subir con README, código Flutter mínimo, prueba widget, workflow, validación de cobertura, plantillas de incidencia y pull request, CODEOWNERS, guía de contribución, backlog, configuración del board, arquitectura y comunicación. Los nombres de usuario y enlaces marcados como USUARIO o URL_DEL_REPOSITORIO deben sustituirse después de crear el repositorio real.

Evidencia	Qué debe mostrar
Repositorio	Árbol de archivos y enlace público o institucional.
CI	Captura de una ejecución verde en develop o pull request.
Board	Vista con las cinco columnas, historias y tareas de Sprint 0.
Comunicación	Captura del servidor, canales, roles e integración de GitHub.
Release	Tag v0.1.0 o v0.1.0-beta.1 y notas de versión.
Pruebas	Resultado de flutter test, cobertura y caso de usabilidad.

Guion de demostración de 3 a 5 minutos

Tiempo	Contenido
0:00-0:40	Problema, usuarios, alcance y advertencia de apoyo.
0:40-1:30	Mapa DevOps y relación entre artefactos.
1:30-2:20	Board, Sprint 0, rama y pull request.
2:20-3:10	Ejecución de CI y prueba Flutter.
3:10-4:00	Discord, canales, normas e integración.
4:00-5:00	Próximos pasos, riesgos y métricas.

Definición de entrega completa

La práctica se considera completa cuando el documento coincide con la configuración real; el repositorio puede clonarse; el workflow se ejecuta; el board contiene el backlog y Sprint 0; las plantillas están disponibles; la herramienta de comunicación tiene canales y normas; y las capturas o enlaces permiten verificar cada punto de la rúbrica.

14. Conclusiones

El plan convierte la propuesta de EVACUA en un proceso de desarrollo controlable. El valor del enfoque DevOps no se limita a automatizar una prueba: relaciona una historia priorizada con código revisado, un artefacto identificable, evidencia de calidad, distribución restringida y observación posterior. Esta trazabilidad es especialmente importante porque la aplicación presenta información vinculada con emergencias.

Sprint 0 reduce incertidumbre antes de ampliar funciones. El repositorio, la integración continua, las reglas de ramas, el board y la comunicación establecen una forma común de trabajar. La arquitectura propuesta permite comenzar con datos simulados y conservar una ruta de crecimiento hacia API, panel administrativo, caché y monitoreo, sin afirmar que el prototipo ya resuelve posicionamiento interior o cálculo dinámico.

La selección de Discord responde a las condiciones de un equipo universitario pequeño y a la necesidad de acceso inmediato, canales, llamadas e integraciones. GitHub seguirá siendo la fuente formal de decisiones técnicas. Conforme EVACUA avance, las metas de cobertura, estabilidad y disponibilidad deberán revisarse con evidencia del piloto, y cualquier publicación de rutas tendrá que mantener aprobación, vigencia y auditoría.

15. Referencias

- [1] Flutter, “Guide to app architecture,” 2026. <https://docs.flutter.dev/app-architecture/guide>
- [2] Flutter, “Testing Flutter apps,” 2026. <https://docs.flutter.dev/testing/overview>
- [3] GitHub, “GitHub Actions documentation,” 2026. <https://docs.github.com/actions>
- [4] GitHub, “About Projects,” 2026. <https://docs.github.com/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects>
- [5] S. Chacon y B. Straub, “Branches in a Nutshell,” Pro Git. <https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell>
- [6] Conventional Commits, “Specification 1.0.0.” <https://www.conventionalcommits.org/en/v1.0.0/>
- [7] T. Preston-Werner, “Semantic Versioning 2.0.0.” <https://semver.org/>
- [8] Firebase, “Firebase App Distribution,” 2026. <https://firebase.google.com/docs/app-distribution>
- [9] Firebase, “Firebase Crashlytics,” 2026. <https://firebase.google.com/docs/crashlytics>
- [10] Firebase, “Firebase Remote Config,” 2026. <https://firebase.google.com/docs/remote-config>
- [11] GitHub, “Using secrets in GitHub Actions,” 2026. <https://docs.github.com/actions/how-tos/write-workflows/choose-what-workflows-do/use-secrets>
- [12] S. Thurgood et al., “Implementing SLOs,” The Site Reliability Workbook. <https://sre.google/workbook/implementing-slos/>
- [13] OWASP, “Mobile Application Security Verification Standard.” <https://mas.owasp.org/MASVS/>
- [14] Slack, “Feature limitations on the free version of Slack,” 2026. <https://slack.com/help/articles/27204752526611-Feature-limitations-on-the-free-version-of-Slack>
- [15] Slack, “Use huddles in Slack,” 2026. <https://slack.com/help/articles/4402059015315-Use-huddles-in-Slack>
- [16] Microsoft, “Free Microsoft Teams,” 2026. <https://www.microsoft.com/microsoft-teams/free>
- [17] Discord, “Beginner’s Guide to Discord,” 2026. <https://support.discord.com/hc/en-us/articles/360045138571-Beginner-s-Guide-to-Discord>
- [18] Discord, “Intro to Webhooks,” 2026. <https://support.discord.com/hc/en-us/articles/228383668-Intro-to-Webhooks>
- [19] Google, “Learn about spaces,” Google Chat Help, 2026. <https://support.google.com/chat/answer/7659784>
- [20] fastlane, “Documentation.” <https://docs.fastlane.tools/>

Enlaces consultados el 17 de septiembre de 2026. Todos los fragmentos de código y diagramas fueron elaborados para EVACUA.